Requirements Document

HTTP 5310 Capstone Project

1. Introduction & Rationale

Develop an AI-powered interactive system that ingests complex case documents (including PDFs with text, images, and graphs), trains them, and interactively guides users on solving the cases through dialog-based AI guidance.

The rationale for this project is twofold:

1. To enable users (students, professionals) to upload case documents containing complex multimodal data and engage with an AI agent that can understand the content, answer questions, and provide stepwise guidance on case analysis and resolution.
2. Solve the problem of manual case document analysis, offering efficient, AI-powered interactive learning tools.
3. Beneficiaries include business, education, and many practice communities.
4. Educational Value – demonstrates full-stack development skills (frontend, backend, database, real-time updates) (Note: modern AI coding technique may be deployed as necessary.
5. User Engagement – this system is interactive and data-driven, ideal for showcasing technical depth and UX.

A core MVP can be delivered using sample case data. Advanced features can be layered after the core loop is stable.

2. Content Evaluation (Main Features of the Project)

2.1 Input Data

Multimodal case document ingestion and parsing (text, photo, graph).

Interactive AI dialog engine using advanced NLP and conversational AI APIs.

Synchronized chat and document viewer with dynamic overlays and highlights.

Guided navigation with AI-generated prompts and progress tracking.

Source: Case Libraries from the internet or proprietary case studies not yet released (e.g. from Longo's school of busines2.4 Supporting Content

- Rules & FAQs (static pages).
- User-generated content (team names, chat messages).

3. Functionality – User Stories

Must Have (MVP)

As a student or User, I want

1. Upload my own case files (PDFs with text, images, graphs) so that the AI can analyze and assist me with solutions.
2. Decide if I want to be instructed/walked through or just ask questions/solutions
3. View the case details in-app
4. Take and save notes using a built-in text editor so that my reflections stay linked to the case.
5. Interact with the interface for questions/advice/guidance
6. Sign in with my account (school account if the user is a student) so my progress and sessions are saved securely
7. Resume previous sessions so that i can continue working from where i left off.

As an instructor or Admin, I want to:

8. Create and manage case libraries so that i can assign cases to specific students or teams.
9. Monitor student activity and progress to evaluate learning outcomes.
10. Ensure content security so that proprietary case files are not shared outside authorized.

Should Have (MVP)

11. Central Case Library where users can browse sample cases before uploading their own.
12. User authentication and profiles for personalized dashboards and saved preferences.
13. Split-screen or multi-window view to see AI chat and document graphics side by side.
14. Session management
15. Instructor dashboard analytics to track class performance and case difficulty patterns.

Nice to Have (MVP)

16. User history and logs for replay over past chats and decisions.
17. Comparison mode to view how different students or teams approached the same case.
18. Real-time collaboration or shared sessions for team-based casework.
19. Business edtion – support for corporate users to analyze confidential internal reports with private LLM deployment.

4. Technical Specifications

Frontend: React app providing document viewer, chat UI, and interaction panels.

Backend: Vibe.d asynchronous server managing WebSocket communication and AI API integration. (TBD)

AI Engines: External dialog APIs for natural language understanding and response generation. (TBD)

Data Layer: Local or cloud storage for documents, user sessions, and interaction logs.

Database: SQL Server or PostgreSQL with Entity Framework Core; optional Redis cache for live queries. (TBD subjected to change)

Data: selected case libaries
Deployment: Frontend on Vercel/Netlify; Backend+DB on Azure/AWS; GitHub + GitHub Actions for CI/CD. (TBD)

Testing: Jest/RTL (frontend), xUnit (backend), CI checks on PR.

Security: password hashing, HTTPS-only, RBAC, input validation.

5. Data Design

5..1 Overview

The system will store user data, uploaded case files, chat interactions, and AI responses in a secure database. The front end communicates with the backend through APIs to retrieve or send information such as user messages, AI responses, and document data.

5.2

Entity	Description	Example Fields
User	Represents a student, instructor or admin	userId, name, email, role
CaseFile	Stores metadata about uploaded PDFs or documents	CaseId, title, fileUrl, uploadedBy, UploadDate
Chatsession	Represents an interaction session between a user and the AI	SessionId, userId, caseId, createdAt
Message	Individual messages in a chat (user or AI)	MessageId, sessionId, sender(user/AI) content, timestamp
Note	User-created text notes linked to a case or chat	NoteId, userId, caseId, content, createdAt

5.3 Data Flow

1. User uploads a case file.

- The file metadata is stored in the database; the file itself is stored in secure object storage. E.g, Aws S3

2. Backend processes and indexes the case.

- Extracted text and embeddings are stored for AI retrieval

3. User interacts with the AI via chat.

- Each question and AI response is logged in the Message table under the current Chatsession.

4. User takes notes or saves progress.

- Notes are stored separately and linked by userId and caseId.

5. Instructor or Admin access.

- Can view assigned cases, monitor usage, or manage uploaded content.

5.4 Data Security & Privacy

- Each user's data (cases, chats, and notes) is **scoped by their account and team**.
- Proprietary documents remain private and are **not used to train external AI models**.
- User authentication ensures only authorized access to sensitive data.
- Role-based permissions control who can upload, view, or comment on cases.

6. Development Specification

6.1 Overview:

The system follows a modern full-stack architecture with a React-based frontend and an API-driven backend powered by AI services. The frontend manages user interaction, chat streaming, and document viewing, while the backend (developed and maintained by the research lead) handles data ingestion, vector retrieval, and model integration.

6.2 Coding Standards

6.2.1 Frontend

- Framework: React + TypeScript
- Code Quality: ESLint and Prettier for consistent formatting
- Structure: Component-driven architecture with reusable UI modules (Chat, PDF Viewer, Notes Panel)
- Styling: Tailwind CSS or Shadcn/UI for consistent design and accessibility

6. Development Specification

6.1 Overview:

The system follows a modern full-stack architecture with a React-based frontend and an API-driven backend powered by AI services. The frontend manages user interaction, chat streaming, and document viewing, while the backend (developed and maintained by the research lead) handles data ingestion, vector retrieval, and model integration.

6.2 Coding Standards

6.2.1 Frontend

- Framework: React + TypeScript
- Code Quality: ESLint and Prettier for consistent formatting
- Structure: Component-driven architecture with reusable UI modules (Chat, PDF Viewer, Notes Panel)
- Styling: Tailwind CSS or Shadcn/UI for consistent design and accessibility

6.2.2 Backend

- We will use AI to design the backend

6.3 Version Control

- Source control managed on **GitHub** using the following workflow:
    - Main branches: main, dev, and feature/* for modular updates.
    - Pull Requests with code reviews before merging.
    - GitHub Projects/Issues used for task tracking and sprint planning.

6.4 Testing

- **Frontend:** Unit and component testing with Jest + React Testing Library for critical flows (chat input, streaming display, and PDF viewer synchronization).
- **Backend:** API integration and functional tests (handled by backend lead).
- **Continuous Integration (CI):** Automated tests run on each pull request before deployment.

6.5 Deployment & Environments

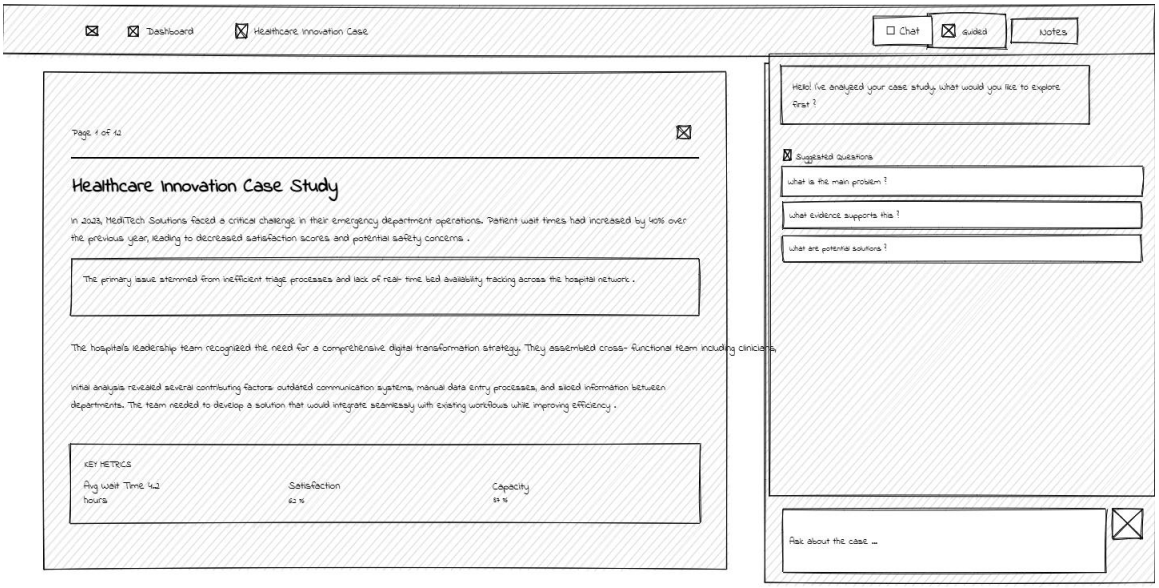
- **Frontend:** Deployed on **Vercel** (or Netlify) for fast CI/CD and global CDN.
- **Backend:** Hosted on **Azure** or **AWS**, connected to a secure database and vector store.
- **Environments:** Separate Development and Production environments with environment variables stored securely.

6.6 Accessibility & Responsiveness

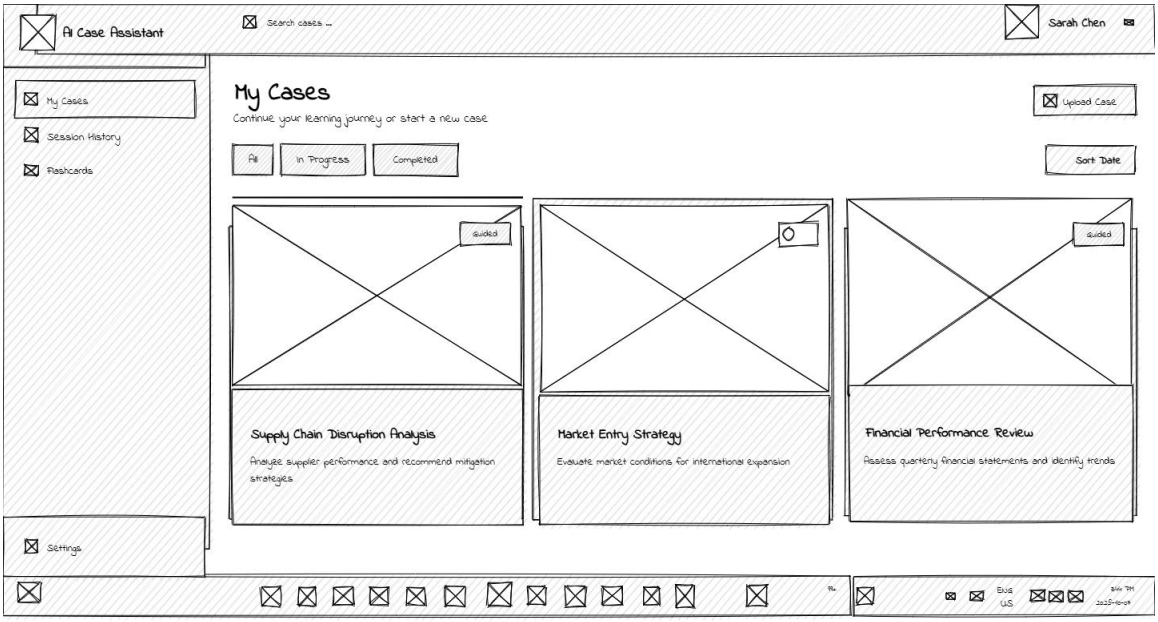
- Conforms to **WCAG 2.1 AA** standards for accessibility.
- Fully responsive layout optimized for desktop, tablet, and mobile.
- Keyboard navigation and ARIA roles are applied for screen readers.

7. Wire Frame

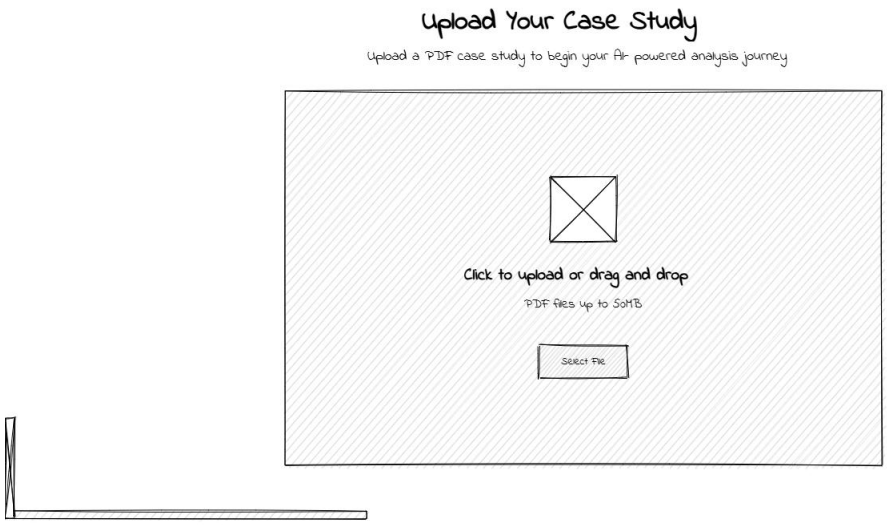
Workspace



Dashboard


AI Case Assistant
Search cases ...
Sarah Chen
My Cases
Session History
Flashcards
Settings
My Cases
Continue your learning journey or start a new case
Upload Case
All
In Progress
Completed
Sort Date
Guided
Supply Chain Disruption Analysis
Analyze supplier performance and recommend mitigation strategies
Market Entry Strategy
Evaluate market conditions for international expansion
Guided
Financial Performance Review
Assess quarterly financial statements and identify trends

Upload


Upload Your Case Study
Upload a PDF case study to begin your AI powered analysis journey
Click to upload or drag and drop
PDF files up to 50MB
Select File

8. Conclusion

The AI Case Tool MVP centers around the parsing of case files, linear/simplified delivery of information, guidance in solving, and research or chatbot assistance. This functionality is packed with an accessible frontend and reliable backend built for secure reliability. This project presents a powerful interface to tackle case problems in a manageable manner and pace with AI integrations ensuring relevance of answers and depth. It acts as assistance to the user, with hints and prompts, prioritizing the testing of problem-solving skills by streamlining the process.